LoanWizard

Apply for a loan by talking to your camera. The model watches, listens, verifies, and prices the offer in minutes, not days: no branch, no paperwork, no waiting room.

Technical Documentation

An AI video loan-origination platform for Indian digital lending: browser perception, a Keras risk and fraud service, a policy engine, and an auditable, replayable decision trail built to the RBI Video-KYC and DPDP 2023 playbook.

Author: Rishet Mehra

Repository: github.com/Rishet11/LoanWizard

Live app: rishet11-loanwizard-web.hf.space

Live ML API: rishet11-loanwizard-ml.hf.space (/health, /docs)

Deck, demo video and PDFs: drive.google.com/drive/folders/1-aanpEn7EELmkbgKpguxsXyxLnKeT2ai

Components: apps/web · apps/ml-service · packages/perception · packages/contracts

Stack: Next.js 14 · FastAPI · TensorFlow / Keras · TensorFlow.js · PostgreSQL

01 The problem, and what we built

Personal loan origination in India still runs on branch visits, photocopied documents, hand-keyed forms, and multi-day waits. Video-KYC made remote onboarding legal, but most products bolt a video call onto the same slow paper process. The decision stays a black box and the audit trail is thin.

LoanWizard makes the interview *itself* the application. A customer opens a web link, grants camera and microphone, and answers a short spoken interview. In the browser, a perception layer transcribes speech, fills the application form, runs passive liveness checks, and reads ID documents. A Python service then scores risk and fraud, applies a policy engine, and returns an instant, explainable offer. Every decision is written with a frozen feature snapshot so it can be replayed and audited later.

Who it serves

Two users, not one. The **borrower** applies by talking to their camera. The **lender or compliance team** needs every decision to be auditable, explainable, and fair. The same flow that prices the loan also produces the evidence a risk team would ask for.

What is genuinely new here

- The interview drives the form by voice; you speak, and name, employment, income, amount and purpose fill in as you talk.
- The decision pipeline is real and explainable end to end, not a scripted screen with a number on it.
- The whole session is wrapped in an auditable, replayable record, down to the exact model version that produced the offer.
- The landing page is an Operator Console that shows the machine working, rather than a scrolling marketing page that describes it.

02 System architecture

The system is a small monorepo with three independently owned surfaces and one shared contract package. Each surface was built as a parallel stream against frozen TypeScript types, so the three could be developed independently and integrated through typed events and a single offer API.

COMPONENT	STREAM	RESPONSIBILITY
packages/perception	A	Browser perception: camera, mic, speech-to-text, TF.js liveness, document OCR, age estimate, device fingerprint. Emits typed `PerceptionEvent`s.
apps/web	C	Next.js 14 App Router: session state machine, Operator Console UI, Prisma audit writes, admin dashboards, internationalisation, server-sent events.
apps/ml-service	B	FastAPI: risk MLP, fraud MLP, persona classifier, policy engine, bureau merge, reason narrator, drift tracker, fairness report, decision replay.
packages/contracts	shared	Frozen TypeScript types, Zod schemas and mock data shared across every stream: this is the integration boundary.

The request path is deliberately **server-authoritative**. The browser never submits the offer inputs directly; it dispatches perception events that the web app persists, and the offer is computed server-side from that persisted data. That single decision is what makes the audit trail trustworthy and form tampering a non-issue.

> **See also.** `docs/architecture-current.pdf` is the as-built diagram for this prototype. `docs/architecture-target.pdf` shows the target production platform (multi-tenant, real disbursement, full compliance) and how the same contract boundary carries through to it.

Why two ORMs over one database

One Postgres database, two object-relational mappers. The web app uses **Prisma** for its session and audit tables; the ML service uses **SQLAlchemy** for its decisions and snapshot tables. They share the same database, which keeps the audit trail in one place while letting each service use the tooling native to its language.

03 The decision pipeline

Each `POST /offer` runs the full chain below and persists the result. None of these steps is faked at runtime; the models are pre-trained and committed under `apps/ml-service/app/models/`.

1. **Bureau merge.** Combines a mock CIBIL and Experian pull into a single credit profile. Mocked behind an adapter (no commercial credentials); deterministic per applicant.
2. **Policy engine.** Hard eligibility rules first: age, income floor, loan cap, liveness, face presence. A failed rule short-circuits to a clear rejection reason.
3. **Risk MLP.** A Keras model (`v1.2.0`) scores the applicant from form fields, the CV summary, bureau data and geo tier, producing a risk band.
4. **Fraud MLP.** A second Keras model (`v0.1.0`) scores fraud from CV signals, device fingerprint and geo.
5. **Persona classifier.** A rules model (`v1.0.0`) labels the applicant, for example `salaried_prime` , to shape the offer and the narrative.
6. **Offer builder.** Maps risk band plus policy result to amount, rate, tenure and EMI. Soft flags (thin file, high loan-to-income) add small, explicit rate adjustments.
7. **Reason narrator.** Produces weighted reason codes and a plain-language explanation: a deterministic template by default, with optional Gemini/Gemma narration.
8. **Audit + snapshot.** Writes the decision row, a frozen input snapshot for replay, the model versions used, and a drift sample for monitoring.

Service surface: `POST /offer` , `GET /health` , `GET /docs` , `GET /fairness/report` , `GET /drift/*` , `POST /transcribe` , `POST /decisions/{id}/replay` .

04 The machine learning

The models are real and reproducible. Synthetic training data is generated by `scripts/generate_synthetic_data.py` and the models are trained with `scripts/train_risk_model.py` and `scripts/train_fraud_model.py` . Both the synthetic set and the training scripts are in the repository, so the committed weights can be regenerated from scratch.

Reason codes that reflect the model, not the inputs

An early version ranked reason codes by raw feature magnitude, which meant income always dominated regardless of what the model had learned. That is a common and misleading shortcut. The current implementation (`app/services/risk_scorer.py`) computes real **permutation importance**. It takes a baseline prediction, then perturbs each of the twelve features to its scaled mean one at a time, re-predicts, and measures the absolute change in output. The whole thing runs as a single batched predict, so it is cheap enough to do on every request. The reason codes therefore reflect what the model actually responds to. The old magnitude heuristic remains only as a fallback when the model is not loaded.

> **Why this matters for judging.** "Explainable AI" is easy to claim and easy to fake. Permutation importance is the difference between an explanation derived from the model and a number dressed up to look like one.

Decision replay on the exact model version

Every decision is stamped with the model versions that produced it (`model_versions_at_decision`), and the weights for each version are archived under `app/models/*/archive/<version>/` . `POST /decisions/{id}/replay` loads the archived risk model matching the decision's recorded version, so a replay reflects the model as it was, not whatever is currently deployed. The response reports `model_version_used` and `exact_model_match` ; if a recorded version is not archived, replay falls back to the current model and flags the mismatch honestly rather than pretending. Training scripts auto-archive each new version, so this stays true as the models evolve.

Fairness and drift

The admin console exposes a fairness report that evaluates outcomes across a cohort file, and a drift tracker that compares live feature distributions against the training baseline. These are the hooks a risk team uses to watch for disparate impact and for the data moving away from what the model was trained on.

05 The browser perception layer

Perception runs in the browser, which keeps the heavy computer-vision work on the client and means raw video never has to leave the device for the core checks. It is a TypeScript package (`packages/perception`) that exposes a single `usePerception()` hook and emits typed events.

Liveness

Passive liveness combines BlazeFace detection and face-landmark tracking to score blink, head-pose movement and a texture signal. The contract includes an optional texture-score field specifically for anti-deepfake scoring, so a stronger model is a drop-in.

Document OCR

On-device Aadhaar and PAN reading with Tesseract.js. No document images are uploaded for the read.

Speech-to-text, with a real fallback

The browser Web Speech API is the primary path. It is reliable on Chrome, weaker on Safari, and disabled by default on Firefox. When confidence is low, the layer automatically falls back to a server-side Whisper endpoint (`POST /transcribe`) wired through `NEXT_PUBLIC_TRANSCRIBE_URL` . The Whisper model is an opt-in deploy toggle to keep the default image light; the fallback wiring is in place regardless.

Age estimate

A real in-browser model via `@vladmandic/face-api` with weights vendored into the repo, degrading gracefully to a placeholder if the weights cannot load so a session never breaks.

> **Honest browser caveat.** Web Speech confidence is weaker on Safari and off by default on Firefox. Use Chrome for the live-camera demo; the Whisper fallback covers the rest.

06 Explainability, auditability and compliance

Compliance is treated as a feature, not a disclaimer. The flow is built to the RBI Video-KYC and DPDP 2023 playbook, and the pieces a regulator or risk team would look for are wired in.

REQUIREMENT	HOW IT IS HANDLED
RBI Video-KYC	Live video interview, liveness checks, and a recording-retention notice. The building blocks are present; a production rollout still needs the lender's own regulatory sign-off.
DPDP Act 2023	Explicit consent capture, disclosed data use, and a right-to-forget endpoint (`/api/consent/[sessionId]/forget`) that deletes a session record.
Explainability	Weighted reason codes plus a plain-language narrative on every offer.
Auditability	Frozen decision snapshots, model versions recorded per decision, one-click replay on the exact archived version.
Fair lending	Cohort fairness report and feature-drift tracking in the admin console.
Borrower protection	A 3-day cool-off surfaced on the accepted screen, per RBI digital-lending guidance, plus a downloadable Key Fact Statement.

Fraud and tamper resistance

Liveness and the fraud MLP defend against someone holding up a photo or a recording. Form tampering and replay are addressed structurally: the form is filled from perception events tied to a server session, and the offer is computed server-side from persisted data, not from anything the client submits, and written with model versions and a frozen snapshot.

07 What is real, and what is mocked

We would rather be precise than oversell. The line below is exactly where the prototype's honesty sits.

> **Real and running.** The full ML decision pipeline (Keras risk MLP with model-derived permutation importance, Keras fraud MLP, rules persona classifier, policy engine, bureau merge, reason narrator); the audit writes and frozen decision snapshots; decision replay on the exact archived model version; the session state machine; the web UI and the admin console (sessions, fairness, drift, replay). Browser perception (TF.js liveness, on-device OCR, the face-api age model, speech, and device fingerprint) is real and runs locally with `NEXT_PUBLIC_USE_MOCK_PERCEPTION=false`.

> **Mocked or optional by design.** The bureau pull (CIBIL/Experian behind an adapter, no live commercial credentials; intentional prototype scope). The LLM narration (deterministic template by default; Gemini/Gemma optional). The Whisper STT model (opt-in deploy toggle; the browser-to-server fallback is wired regardless). The public judge link runs scripted perception and a reliable mock offer so the end-to-end demo never depends on a webcam or a cold ML Space.

The bureau pull is the one substantial integration left mocked, and that is a deliberate scoping decision: live CIBIL or Experian access needs commercial credentials, not more code. The adapter interface is in place (`app/services/bureau/`), so a real pull is a drop-in.

08 Running the prototype

Requirements: Node 18+, pnpm 8, Python 3.11, and either Docker or a local PostgreSQL 16+. The web build needs no database. All routes are server-rendered on demand, so a clean checkout type-checks and builds without any secrets.

```
# 1. Install JS dependencies
pnpm install

# 2. Bring up Postgres + the ML service
docker compose up -d

# 3. Create the web database tables
cd apps/web
DATABASE_URL=postgresql://loan:loan@localhost:5432/loan pnpm exec prisma db push

# 4. Start the web app, then open http://localhost:3000
pnpm --filter @loan-wizard/web dev
```

If Docker is unavailable, the ML service runs natively (Python virtualenv + `uvicorn`); the README documents that path. To drive the flow without a camera, set `NEXT_PUBLIC_USE_MOCK_PERCEPTION=true`, which replays canned perception events. This is how the hosted public demo runs.

> **Verified in a clean environment.** Web: `tsc --noEmit` clean and `next build` succeeds with no `DATABASE_URL`. Perception: 33/33 Vitest tests pass. ML service: 105 pytest tests pass; `POST /offer` against the committed Keras models returns an eligible offer with risk band, fraud score, three reason codes and a narrative, with model versions stamped.

Deployment

The recommended hosted shape is two Docker Spaces on Hugging Face: the repository root for the web app (port 3000) and `apps/ml-service` for the ML service (port 8000), with Neon Postgres for the shared audit trail. `ADMIN_PASSWORD` is required on the web Space and `DATABASE_URL` on both. The full runbook is in `docs/DEPLOYMENT.md`.

09 Technology stack

LAYER	TECHNOLOGIES
Frontend	Next.js 14 (App Router), React 18, Tailwind v4 with CSS-variable design tokens, Framer Motion, next-intl
Perception	TensorFlow.js, BlazeFace, face-landmarks-detection, @vladmandic/face-api (age), Tesseract.js, Web Speech API
ML service	FastAPI, TensorFlow / Keras, scikit-learn, pandas, SQLAlchemy; optional faster-whisper, optional Gemini/Gemma
Data	PostgreSQL; Prisma (web) and SQLAlchemy (ML service) over one database
Tooling	pnpm workspaces, Turborepo, TypeScript, Zod, pytest, Docker
Hosting	Hugging Face Docker Spaces (web + ML), Neon Postgres

Scalability

The ML service is stateless per request and scales horizontally behind a load balancer. Perception runs in the browser, so the heavy CV work is on the client and does not consume server capacity. Postgres holds the shared state; nothing in the request path keeps session state in memory. The `architecture-target.pdf` diagram shows how the same contract boundary extends to a multi-tenant platform with an event spine, a feature store and model serving.

10 Known limitations and roadmap

As of this prototype, and stated plainly:

- **Bureau pull is mocked** behind an adapter. This is intentional scope: live access needs commercial credentials, and a real adapter is a drop-in.
- **Server-side Whisper is off by default** to keep the image light; the browser-to-server STT fallback is wired and the model is a one-line deploy toggle.
- **Age estimation** runs a real in-browser model but degrades to a placeholder if the weights fail to load, so a session never breaks.
- **The public judge link** runs scripted perception and a reliable mock offer; the real camera and ML path runs locally.

Next: live bureau integrations behind the existing adapter, deepfake-resistant liveness using the optional texture model, a lender console for human-in-the-loop review on borderline decisions, and the target platform build-out (disbursement, servicing, co-lending) along the roadmap in the target architecture.